

Documentación Técnica

Panini Support — Prueba de Concepto Móvil

Gestión de tickets de soporte · Álbum oficial Copa Mundial FIFA 2026

Documento orientado a ingenieros que continúen el desarrollo. Describe las decisiones de diseño, la estrategia reactiva, los Feature Flags y el flujo general del sistema, conectados directamente con el código implementado.

1. Justificación técnica

El objetivo de la PoC es reemplazar un flujo de soporte disperso (correos, hojas de cálculo, mensajes informales) por una app móvil centralizada, reactiva y mantenible, dentro de un alcance acotado (sin backend real). Las tecnologías se eligieron por ser el estándar actual de Android y por ajustarse al alcance:

- **Jetpack Compose + Material 3:** UI declarativa; la pantalla se redibuja sola ante cambios de estado, habilitando la reactividad sin código manual de refresco.
- **MVVM:** separa la lógica (ViewModel) de la presentación (Composable) y de los datos (Repository). La UI queda “tonta” y testeable.
- **Coroutines + StateFlow/SharedFlow:** asincronía y reactividad nativas de Kotlin, sin librerías externas.
- **Retrofit:** estándar de facto para consumo HTTP; se deja configurado para la integración futura.
- **DI manual (Service Locator):** suficiente para el alcance; evita el overhead de Hilt en una PoC.

Criterio rector: **simplicidad justificada**. No se incorporan patrones ni librerías que no aporten valor real al alcance actual.

2. Decisiones arquitectónicas

2.1 Capas

- **domain/model:** modelos puros (Ticket, Priority, TicketStatus, TicketCategory). No dependen de red ni de UI.
- **data:** DTOs (separados en request/response), mappers, datos mock, contrato Retrofit y repositorios. Traduce el mundo externo al dominio.
- **ui:** pantallas Compose y ViewModels, organizados por feature.
- **core / di:** utilidades transversales (estado, eventos, flags) e inyección.

2.2 Organización por feature

Cada pantalla agrupa su Screen y su ViewModel en el mismo paquete (ui/login, ui/ticketlist, etc.) en lugar de separar por tipo técnico. Esto mejora la cohesión: un ingeniero que modifica una pantalla encuentra todo en un solo lugar.

2.3 Programación contra interfaces

Los repositorios se exponen como interfaces (ITicketRepository, IAuthRepository) y se implementan en clases concretas (TicketRepository, AuthRepository). Los ViewModels dependen de la interfaz, nunca de la implementación. Consecuencia: migrar a un backend real (o a Hilt) solo requiere cambiar la instancia provista en AppContainer, sin tocar la UI ni los ViewModels.

2.4 Inyección de dependencias

AppContainer es un Service Locator (un object singleton) que provee el repositorio de tickets, el de autenticación y el bus de eventos como instancias únicas. La unicidad es crítica: un solo TicketRepository compartido garantiza que la “única fuente de verdad” sea realmente única entre todas las pantallas. AppViewModelFactory construye los ViewModels inyectándoles estas dependencias.

2.5 Manejo de estados

Todas las pantallas usan UiState (sealed interface) con tres casos: Loading, Success<T> y Error. Al ser sellada, el when en la UI obliga a cubrir los tres estados. Reemplaza el manejo disperso de banderas booleanas y evita estados inconsistentes.

2.6 DTOs separados por intención

CreateTicketRequest (entrada) y TicketResponse (salida) son tipos distintos. El request no incluye id, status ni createdAt porque los genera el servidor; el tipo refleja esa regla de negocio y evita errores en tiempo de compilación. El mapeo DTO ↔ dominio vive en data/mapper.

3. Estrategia basada en eventos

El sistema distingue deliberadamente entre **estado** y **eventos**, y usa un mecanismo distinto para cada uno.

3.1 Estado — StateFlow

TicketRepository mantiene un MutableStateFlow<List<Ticket>> privado y expone su versión de solo lectura (tickets). Es la única fuente de verdad del listado.

- Tiene siempre un valor actual y lo reentrega al recomponer o al suscribirse un nuevo observador.
- Toda mutación (crear, actualizar estado/prioridad) genera una **lista nueva** (inmutabilidad con copy() y update {}) y la emite.

- Los ViewModels colectan este flujo; la UI lo observa con `collectAsStateWithLifecycle()` y se redibuja automáticamente.

Esto resuelve los dos escenarios obligatorios:

- **Creación de tickets:** `createTicket` agrega el ticket y emite la lista nueva. El listado, suscripto, lo muestra sin recargar.
- **Cambio de prioridad:** `updatePriority` modifica el ticket y reordena (`sortedForBoard`, por peso de prioridad descendente). El reposicionamiento emerge del reordenamiento, sin lógica especial de “mover”.

3.2 Eventos — `SharedFlow`

`AppEventBus` usa un `SharedFlow` para notificaciones transitorias (`AppEvent.TicketCreated`, `PriorityUpdated`, `StatusUpdated`).

- A diferencia del estado, un evento se consume **una sola vez** y no tiene “valor actual”: no se reentrega al rotar la pantalla. Esto evita el bug clásico del `snackbar` duplicado.
- El repositorio emite eventos tras cada mutación, **sin conocer** quién los consume. `TicketListViewModel` los expone y la pantalla los muestra como `snackbar` mediante un `LaunchedEffect`.

3.3 Por qué dos mecanismos

Al crear un ticket ocurren dos cosas por dos canales: el listado se actualiza (`StateFlow`, estado persistente) y aparece un aviso (`SharedFlow`, evento transitorio). Usar un único mecanismo para ambos produciría o eventos repetidos o un listado que no persiste. La elección correcta por caso es la decisión arquitectónica central de esta PoC.

4. Feature Flags

Centralizados en `core/featureflags/FeatureFlags.kt` como un object con constantes booleanas. La fuente de verdad es un único archivo: cambiar una funcionalidad implica modificar una sola línea.

Flag	Tipo de control	Efecto en la UI
<code>CREATE_TICKET_ENABLED</code>	Ocultar	El botón de creación no se dibuja
<code>PRIORITY_UPDATE_ENABLED</code>	Deshabilitar	El selector de prioridad queda inactivo
<code>SHOW_ADMIN_CATEGORIES</code>	Filtrar	Las categorías <code>adminOnly</code> no aparecen

Los flags se consumen directamente en la capa UI, ya que controlan decisiones de presentación (qué mostrar o permitir). No afectan ViewModels ni repositorio.

Evolución futura: como toda la app lee a través de FeatureFlags, el objeto puede reemplazarse por una fuente remota (Remote Config, endpoint /config) sin modificar los puntos de uso. En esta PoC son const (fijos en compilación), lo cual es suficiente; si se requiriera cambio en runtime, dejarían de ser const.

5. Flujo general del sistema

5.1 Flujo de datos (lectura)

Repository (StateFlow) → ViewModel (UiState) → UI (collectAsStateWithLifecycle)

El repositorio es la fuente; el ViewModel transforma la lista en UiState; la UI observa y dibuja. La UI no contiene lógica de negocio.

5.2 Flujo de acción (escritura)

UI (acción) → ViewModel → Repository (muta y emite) → StateFlow + Evento

Ejemplo (crear ticket):

- El usuario completa el formulario y pulsa “Crear”.
- CreateTicketViewModel.create() valida y llama a repository.createTicket().
- El repositorio agrega el ticket, emite la lista nueva por StateFlow y un evento por SharedFlow.
- La pantalla vuelve al listado (popBackStack), que ya refleja el ticket nuevo (StateFlow) y muestra el snackbar (SharedFlow).

5.3 Navegación

AppNavGraph centraliza el ruteo. Pantallas: Login → Lista → (Detalle | Crear). Al iniciar sesión se elimina el Login del back-stack (popUpTo(LOGIN){inclusive=true}). El detalle recibe el ticketId como argumento de ruta tipado. Las pantallas reciben lambdas de navegación, no el NavController, para mantenerse desacopladas.

6. Integración futura del backend

La capa de red está preparada pero inactiva (la PoC usa mock). Para conectar el backend real:

- Ajustar RetrofitClient.BASE_URL con la URL del servicio.
- Implementar ITicketRepository consumiendo TicketApi (Retrofit) y mapeando los DTOs a dominio con toDomain().
- Cambiar la instancia provista en AppContainer por esa nueva implementación.

Ningún otro punto del proyecto requiere cambios: ViewModels, UI y navegación dependen de la interfaz, no de la fuente de datos. El contrato de los endpoints está documentado en `/contracts/tickets-api.yaml`, coherente con TicketApi.